

Recordar

- 1. Qual é a diferença essencial entre um broker e um mediator?** O broker apenas roteia mensagens entre produtor e consumidor, sem tomar decisões de negócio. Já o mediator conhece e coordena todo o processo, definindo a sequência dos passos.
- 2. Como se nomeia corretamente um evento? E um comando? Dê um exemplo de cada, usando o domínio da situação acima.** Um evento usa o particípio passado por representar um fato já ocorrido (ex: `ResultadoLaboratorialDisponibilizado`). Um comando usa o infinitivo ou ordem direta por pedir uma ação (ex: `GerarCobranca`).
- 3. Classifique cada um dos seis nomes da situação (`ResultadoLaboratorialDisponibilizado.v1`, `GerarCobranca`, `hospital.events`, `billing.resultados.v1`, `hospital.events.dlx` e `billing.resultados.v1.dlq`) numa das categorias evento, comando, exchange de domínio, fila de trabalho, exchange de dead-letter ou fila de dead-letter.** `ResultadoLaboratorialDisponibilizado.v1` é evento; `GerarCobranca` é comando; `hospital.events` é exchange de domínio; `billing.resultados.v1` é fila de trabalho; `hospital.events.dlx` é exchange de dead-letter; e `billing.resultados.v1.dlq` é fila de dead-letter.
- 4. Defina, numa frase cada, entrega pelo menos uma vez, idempotência, ordenação e dead-letter queue.** **Entrega pelo menos uma vez:** a mensagem pode chegar repetida. **Idempotência:** processar a mensagem repetida não duplica o efeito de negócio. **Ordenação:** garante a sequência correta dos eventos. **Dead-letter queue:** fila para mensagens rejeitadas, evitando que sejam perdidas.

Compreender

- 1. Descreva, em uma sequência de passos concreta, o que acontece quando o consumidor de Faturamento grava o efeito de uma cobrança no SQLite e o processo cai antes de confirmar a mensagem ao RabbitMQ. Explique o que o broker faz a seguir com aquela mensagem e por que ele não tem nenhuma forma de saber que o efeito já havia sido registrado do outro lado.** O consumidor grava os dados no banco, mas cai antes de confirmar (enviar o ack) ao RabbitMQ. Como o broker não acessa o banco do consumidor, ele só sabe que não recebeu a confirmação. Por precaução, ele reentrega a mensagem quando o sistema volta.
- 2. A pessoa da situação, que propôs ignorar todos os redeliveries, está tentando resolver um problema real com a ferramenta errada. Explique por que a entrega repetida é uma consequência necessária de garantir que nenhuma mensagem se perca diante de falhas ambíguas como a do item anterior, e diga o que a equipe perderia de fato se simplesmente desligasse o redelivery em vez de tratar a repetição.** O broker não sabe se a confirmação falhou na rede ou se o consumidor não processou a mensagem. A reentrega é a escolha mais segura para evitar perda de dados. Desligar o redelivery trocava a duplicidade (que a idempotência resolve) pela perda silenciosa de resultados de exames.
- 3. No log do consumidor de Faturamento aparece a linha `processed=False attempts=2` para um `event_id` que já tinha gerado efeito antes, mas a tabela `billing_effects` mostra só uma linha de cobrança para essa mesma identidade. Um colega, olhando só a linha do log, conclui que há um bug porque "a mensagem foi processada duas vezes". Usando esse log e essa tabela como evidência, diferencie tentativa, confirmação e efeito de negócio, e explique por que a conclusão do colega está**

errada. Tentativa é quantas vezes o consumidor recebeu a mensagem; **confirmação** é o ack enviado ao broker; **efeito de negócio** é o registro real no banco (que ocorre só uma vez). O colega confundiu tentativa com efeito. Ter `attempts=2` no log e só um registro no banco é a prova exata de que a idempotência funcionou.

4. Alguém na equipe propõe resolver a duplicidade guardando os `event_id` já vistos num `set()` em memória, dentro do próprio processo Python do consumidor, em vez de gravar cada tentativa no SQLite. Explique por que essa alternativa falha na primeira vez que o serviço reinicia ou que uma segunda réplica do consumidor entra no ar, e diga o que ter `event_id` como chave primária durável resolve que o `set()` em memória não resolve. Um `set()` em memória é apagado ao reiniciar o serviço e não é compartilhado se houver mais de uma réplica rodando. Salvar o `event_id` como chave primária no banco torna o histórico durável e acessível por qualquer réplica, resolvendo o problema de forma definitiva.